

INFORME DE LABORATORIO N°3

Antony Brandon Quispe Calizaya
Sistemas Operativos

Índice

1. Introducción	4
2. Objetivos	4
2.1. Objetivo General	4
2.2. Objetivos Específicos	4
3. Marco Teórico	5
3.1. Gestión de Memoria	5
3.2. Fragmentación	5
3.2.1. Fragmentación Interna	5
3.2.2. Fragmentación Externa	5
4. Particiones Fijas	5
4.1. Definición	5
4.2. Ventajas	5
4.3. Desventajas	5
4.4. Implementación	6
4.5. Asignación de Procesos	6
5. Particiones Dinámicas	6
5.1. Definición	6
5.2. Ventajas	6
5.3. Desventajas	6
5.4. Inicialización	7
5.5. División de Bloques	7
6. Particiones Dinámicas con Compactación	7
6.1. Definición	7
6.2. Ventajas	7
6.3. Desventajas	8
6.4. Implementación	8
7. Algoritmo de Gemelos (Buddy System)	8
7.1. Definición	8
7.2. Funcionamiento	8
7.3. Obtención de Potencia de 2	9
7.4. División de Bloques	9
7.5. Fusión de Gemelos	9
8. Algoritmos de Asignación	9
8.1. First Fit	9
8.1.1. Implementación	9
8.1.2. Ventajas	10
8.1.3. Desventajas	10
8.2. Best Fit	10
8.2.1. Implementación	10
8.2.2. Ventajas	10

8.2.3. Desventajas	10
8.3. Worst Fit	10
8.3.1. Implementación	10
8.3.2. Ventajas	10
8.3.3. Desventajas	10
9. Análisis General de la Implementación	11
9.1. Diseño Modular	11
9.2. Lista de Bloques	11
9.3. Liberación y Unión de Memoria	11
9.4. Ventajas de la Implementación	11
9.5. Limitaciones	11
10.Conclusiones	12
11.Código Completo Implementado	12
12.Repositorio del Proyecto	15

1. Introducción

La gestión de memoria es una de las funciones fundamentales de un sistema operativo, ya que permite administrar eficientemente la memoria RAM para ejecutar múltiples procesos de manera concurrente.

En este informe se analizan distintos algoritmos de asignación de memoria implementados en Python mediante la clase `GestorMemoria`. Se estudian los siguientes métodos:

- Particiones fijas
- Particiones dinámicas
- Particiones dinámicas con compactación
- Algoritmo de gemelos (Buddy System)

Además, se analizan los algoritmos de selección de bloques:

- First Fit
- Best Fit
- Worst Fit

Finalmente, se explica detalladamente cómo cada algoritmo fue implementado dentro del código proporcionado.

2. Objetivos

2.1. Objetivo General

Analizar e implementar distintos algoritmos de gestión de memoria utilizados en sistemas operativos.

2.2. Objetivos Específicos

- Comprender el funcionamiento de las particiones fijas y dinámicas.
- Analizar el problema de fragmentación interna y externa.
- Estudiar el funcionamiento de la compactación de memoria.
- Implementar y analizar el algoritmo de gemelos.
- Comparar los algoritmos First Fit, Best Fit y Worst Fit.

3. Marco Teórico

3.1. Gestión de Memoria

La gestión de memoria es el proceso mediante el cual el sistema operativo administra la memoria RAM disponible para los procesos.

Cada proceso necesita una cantidad determinada de memoria para ejecutarse, por lo que el sistema debe decidir:

- Dónde almacenar cada proceso.
- Cómo reutilizar memoria liberada.
- Cómo minimizar desperdicio de memoria.

3.2. Fragmentación

La fragmentación representa memoria desperdiciada durante la asignación.

3.2.1. Fragmentación Interna

Ocurre cuando un proceso utiliza menos memoria que el tamaño del bloque asignado.

Ejemplo:

- Bloque asignado: 256 KB
- Proceso: 200 KB
- Fragmentación interna: 56 KB

3.2.2. Fragmentación Externa

Ocurre cuando existen pequeños espacios libres dispersos en memoria que no pueden utilizarse eficientemente.

4. Particiones Fijas

4.1. Definición

En este esquema, la memoria se divide previamente en bloques de tamaño fijo.

Cada partición puede almacenar únicamente un proceso.

4.2. Ventajas

- Implementación sencilla.
- Administración rápida.

4.3. Desventajas

- Alta fragmentación interna.
- Limitación en el número de procesos simultáneos.

4.4. Implementación

Las particiones se crean en el constructor:

```
if tipo_memoria == "Fijas":  
    inicio = 0  
  
    for tam in particiones_fijas:  
        self.bloques.append(  
            BloqueMemoria(inicio, tam)  
        )  
  
        inicio += tam
```

4.5. Asignación de Procesos

```
bloque.libre = False  
  
bloque.proceso = proceso  
  
bloque.fragmentacion = (  
    bloque.tamano - proceso.memoria  
)
```

La fragmentación interna se calcula mediante:

$$Fragmentacion = TamanoBloque - TamanoProceso$$

5. Particiones Dinámicas

5.1. Definición

La memoria no se divide previamente. Los bloques son creados dinámicamente según las necesidades de cada proceso.

5.2. Ventajas

- Menor fragmentación interna.
- Mejor aprovechamiento de memoria.

5.3. Desventajas

- Produce fragmentación externa.

5.4. Inicialización

Inicialmente existe un único bloque libre:

```
self.bloques.append(  
    BloqueMemoria(0, total_ram)  
)
```

5.5. División de Bloques

```
nuevo = BloqueMemoria(  
    bloque.inicio,  
    proceso.memoria,  
    libre=False,  
    proceso=proceso  
)
```

Cálculo del espacio restante:

```
restante = (  
    bloque.tamano - proceso.memoria  
)
```

Creación del bloque libre restante:

```
if restante > 0:  
  
    libre = BloqueMemoria(  
        nuevo.inicio + proceso.memoria,  
        restante  
    )  
  
    self.bloques.insert(  
        indice + 1,  
        libre  
    )
```

6. Particiones Dinámicas con Compactación

6.1. Definición

La compactación reorganiza los bloques ocupados para unir toda la memoria libre en una sola región continua.

6.2. Ventajas

- Reduce fragmentación externa.
- Permite reutilizar memoria dispersa.

6.3. Desventajas

- Alto costo computacional.
- Reubicación constante de procesos.

6.4. Implementación

Si no existe memoria suficiente:

```
if not libres:

    if self.tipo_memoria == "Dinamicas Compactacion":

        self.compactar()
```

Compactación de bloques:

```
for b in ocupados:

    nuevo = BloqueMemoria(
        inicio,
        b.tamano,
        libre=False,
        proceso=b.proceso
    )

    nuevos.append(nuevo)

    inicio += b.tamano
```

Creación del bloque libre final:

```
libre_total = self.total_ram - inicio
```

7. Algoritmo de Gemelos (Buddy System)

7.1. Definición

El algoritmo de gemelos divide la memoria en bloques cuyo tamaño corresponde a potencias de 2.

7.2. Funcionamiento

1. Se busca un bloque libre adecuado.
2. Si el bloque es demasiado grande, se divide en dos mitades iguales.
3. La división continúa hasta obtener el tamaño mínimo suficiente.
4. Cuando un bloque se libera, puede fusionarse con su bloque gemelo.

7.3. Obtención de Potencia de 2

```
tam_requerido = 1

while tam_requerido < proceso.memoria:
    tam_requerido *= 2
```

7.4. División de Bloques

```
while (
    bloque.tamano // 2 >= tam_requerido
):
```

Creación de bloques gemelos:

```
bloque1 = BloqueMemoria(
    bloque.inicio,
    mitad
)

bloque2 = BloqueMemoria(
    bloque.inicio + mitad,
    mitad
)
```

7.5. Fusión de Gemelos

Verificación de buddies:

```
if a.inicio ^ a.tamano == b.inicio:
```

Fusión:

```
nuevo = BloqueMemoria(
    min(a.inicio, b.inicio),
    a.tamano * 2
)
```

8. Algoritmos de Asignación

8.1. First Fit

Selecciona el primer bloque disponible que sea suficientemente grande.

8.1.1. Implementación

```
if self.algoritmo == "First Fit":

    bloque = libres[0]
```

8.1.2. Ventajas

- Muy rápido.
- Bajo costo computacional.

8.1.3. Desventajas

- Incrementa fragmentación externa.

8.2. Best Fit

Selecciona el bloque más pequeño posible que pueda almacenar el proceso.

8.2.1. Implementación

```
bloque = min(  
    libres,  
    key=lambda b: b.tamano  
)
```

8.2.2. Ventajas

- Reduce desperdicio inmediato.

8.2.3. Desventajas

- Genera bloques libres pequeños.

8.3. Worst Fit

Selecciona el bloque libre más grande disponible.

8.3.1. Implementación

```
bloque = max(  
    libres,  
    key=lambda b: b.tamano  
)
```

8.3.2. Ventajas

- Mantiene bloques libres grandes.

8.3.3. Desventajas

- Puede desperdiciar memoria.

9. Análisis General de la Implementación

9.1. Diseño Modular

La implementación utiliza programación orientada a objetos mediante:

- Clase `GestorMemoria`
- Clase `BloqueMemoria`

Esto permite reutilizar y extender funcionalidades fácilmente.

9.2. Lista de Bloques

Toda la memoria es administrada mediante:

```
self.bloques
```

Cada elemento representa una región continua de memoria.

9.3. Liberación y Unión de Memoria

Cuando un proceso termina:

```
b.libre = True  
  
b.proceso = None  
  
b.fragmentacion = 0
```

Posteriormente se fusionan bloques contiguos:

```
self.unir_libres()
```

9.4. Ventajas de la Implementación

- Código modular y extensible.
- Fácil visualización del estado de memoria.
- Permite comparar múltiples algoritmos.
- Simula correctamente fragmentación y compactación.

9.5. Limitaciones

- No implementa memoria virtual.
- No existe paginación.
- No implementa swapping.
- Compactación posee alto costo computacional.

10. Conclusiones

- Las particiones fijas son simples, pero generan alta fragmentación interna.
- Las particiones dinámicas aprovechan mejor la memoria, aunque generan fragmentación externa.
- La compactación mejora el aprovechamiento de memoria al unir espacios dispersos.
- El algoritmo de gemelos proporciona divisiones y fusiones eficientes utilizando bloques potencia de 2.
- First Fit prioriza velocidad, Best Fit minimiza desperdicio inmediato y Worst Fit intenta preservar bloques grandes.
- La implementación desarrollada permite comparar diferentes estrategias de asignación de memoria de manera modular y flexible.

11. Código Completo Implementado

```
from models.bloque_memoria import BloqueMemoria

class GestorMemoria:

    def __init__(
        self,
        total_ram,
        tipo_memoria,
        algoritmo,
        particiones_fijas=None
    ):

        self.total_ram = total_ram

        self.tipo_memoria = tipo_memoria

        self.algoritmo = algoritmo

        self.bloques = []

        if tipo_memoria == "Fijas":

            inicio = 0

            for tam in particiones_fijas:

                self.bloques.append(
                    BloqueMemoria(inicio, tam)
                )
```

```
        inicio += tam

    else:

        self.bloques.append(
            BloqueMemoria(0, total_ram)
        )

def asignar(self, proceso):

    libres = [
        b for b in self.bloques
        if b.libre and b.tamano >= proceso.memoria
    ]

    if not libres:

        if self.tipo_memoria == "Dinamicas Compactacion":

            self.compactar()

            libres = [
                b for b in self.bloques
                if b.libre and b.tamano >= proceso.memoria
            ]

            if not libres:
                return False

        if self.tipo_memoria == "Gemelos":

            bloque = min(
                libres,
                key=lambda b: b.tamano
            )

            tam_requerido = 1

            while tam_requerido < proceso.memoria:
                tam_requerido *= 2

            while (
                bloque.tamano // 2 >= tam_requerido
            ):

                mitad = bloque.tamano // 2

                indice = self.bloques.index(bloque)

                bloque1 = BloqueMemoria(
                    bloque.inicio,
```

```
        mitad
    )

    bloque2 = BloqueMemoria(
        bloque.inicio + mitad,
        mitad
    )

    self.bloques.pop(indice)

    self.bloques.insert(indice, bloque2)

    self.bloques.insert(indice, bloque1)

    bloque = bloque1

    bloque.libre = False

    bloque.proceso = proceso

    bloque.fragmentacion = (
        bloque.tamano - proceso.memoria
    )

    return True

if self.algoritmo == "First Fit":

    bloque = libres[0]

elif self.algoritmo == "Best Fit":

    bloque = min(
        libres,
        key=lambda b: b.tamano
    )

else:

    bloque = max(
        libres,
        key=lambda b: b.tamano
    )

if self.tipo_memoria == "Fijas":

    bloque.libre = False

    bloque.proceso = proceso

    bloque.fragmentacion = (
```

```
        bloque.tamano - proceso.memoria
    )

    return True

nuevo = BloqueMemoria(
    bloque.inicio,
    proceso.memoria,
    libre=False,
    proceso=proceso
)

restante = (
    bloque.tamano - proceso.memoria
)

indice = self.bloques.index(bloque)

self.bloques.pop(indice)

self.bloques.insert(indice, nuevo)

if restante > 0:

    libre = BloqueMemoria(
        nuevo.inicio + proceso.memoria,
        restante
    )

    self.bloques.insert(
        indice + 1,
        libre
    )

return True
```

12. Repositorio del Proyecto

El código fuente completo del proyecto se encuentra disponible en GitHub:

Repositorio Oficial del Proyecto